

UMI 两端测序不准校正方案

王涵一 2200012109

目录

1. 实验背景	1
2. 实验方法	2
2.1. 白名单索引	2
2.2. 经典局部比对 (Smith-Waterman 算法)	2
2.3. 神经网络	2
3. 实验结果	3
3.1. 输入数据	3
3.2. 校正流水线	6
3.3. 校正结果	13
3.4. 算法时间复杂度计算	17
4. 讨论	18
5. 参考文献	18

1. 实验背景

单细胞转录组测序 (scRNA-seq) 技术的发展,使我们能够在单个细胞的精度上解析复杂的生物系统,但其准确性严重依赖于对起始 RNA 分子的精确定量。由于起始材料的稀缺性,PCR 扩增成为文库构建中不可或缺的一步,然而这也引入了扩增偏好和错误,对定量造成干扰。为了校正这些偏差,唯一分子标识符 (Unique Molecular Identifier, UMI) 技术被广泛应用,它通过为每个原始转录本标记上独特的短 DNA 序列,使得我们可以通过计算独特 UMI 的数量来推断原始分子的数量。[1]

然而,UMI 技术在实践中并非完美无缺。测序过程本身会引入错误,导致一个原始 UMI 衍生出大量仅有微小差异的错误拷贝。这些错误并非均匀分布,一个常见的现象是 UMI 读段两端的碱基质量偏低,错误率显著更高。此外,测序过程还可能引入插入和缺失 (Indel) 错误,导致 UMI 读段的长度发生变化。这两种复杂的错误模式——位置依赖的替换错误和 Indel——使得传统的、基于全局汉明距离的校正方法效果不佳,它们难以区分真正的低丰度新分子和由高丰度分子产生的、带有多个错误的衍生体。

因此,一个鲁棒的 UMI 校正算法必须能够智能地应对这些挑战。它需要具备区分替换错误和 Indel 错误的能力,并且在处理替换错误时,能够学习并适应错误在序列上非均匀分布的模式。简单、单一的校正策略已不足以应对这种复杂性。本研究的目标正是设计并实现一个多阶段的混合式校正流水线,该流程首先通过快速索引处理绝大多数简单错误,然后利用专门的局部比对算法处理 Indel,最后借助深度学习模型强大的模式识别能力,来解决剩余的、包含多个或复杂位

置错误的疑难 UMI。通过这种分而治之的策略，旨在从充满噪声的测序数据中，高效且准确地还原出真实的分子计数。

2. 实验方法

本实验主要采用的方法为白名单索引、经典局部比对和神经网络深度学习。

2.1. 白名单索引

白名单索引是一种高效的预计算查找策略，旨在快速校正那些与真实 UMI 仅有单个替换错误的读段。由于 PCR 和测序过程中，大部分错误都是单个碱基的替换，这种错误模式最为常见。该方法通过预先构建一个哈希表，将所有白名单 UMI 的单点突变体（即汉明距离为 1 的序列）作为 key，原始白名单 UMI 作为 value 进行存储。在校正时，对于一个未知的 UMI 序列，算法可以直接通过哈希查找，以 $O(L)$ 的时间复杂度（ L 为 UMI 长度）快速找到其对应的原始序列，从而避免了耗时的逐一比对。这种方法极大地提升了处理海量简单错误时的效率，为后续更复杂的算法模块减轻了计算负担。

2.2. 经典局部比对（Smith-Waterman 算法）

全局比对算法要求对整个序列进行从头到尾的比对。这对于长度差异大或只在局部区域有相似性的序列来说，效果往往不尽如人意。而 UMI 的全局比对效果不佳正是在 PCR 和测序过程中可能出现的 Indel，使得只有局部区域有相似性。

局部比对则专注于找出两个序列之间相似度最高的一个或多个子序列，而忽略序列的其他部分。Smith-Waterman 算法是局部序列比对的一个经典算法。它利用动态规划，通过构建和填充一个得分矩阵，初始化第一行/列为 0、递归计算取最大值且不小于 0，高效地找出两个序列之间相似度最高的一个或多个连续片段。

2.3. 神经网络

神经网络是受到人类大脑神经元网络结构启发而设计的一种计算模型。它由一系列相互连接的节点（称为神经元）组成，这些节点按照层次结构排列，通常分为输入层、隐藏层和输出层。每个神经元都与前一层的所有神经元连接，并且每条连接都有一个权重，用于调节信号的传递。前向传播主要为输入数据通过神经网络的输入层，沿着连接传递到隐藏层和输出层，并最终产生输出。在传播过程中，每个神经元执行加权求和和激活函数操作，将输入信号转换为输出信号。而反向传播为通过与期望输出进行比较，计算输出误差。然后，误差以反向传播的方式沿着网络传播回来，用于调整连接权重，以减小误差。这通常使用梯度下降等优化算法来完成。

训练过程主要包括：

- 初始化-初始化神经网络的权重和偏置参数。
- 前向传播-将训练数据输入到网络中，计算输出结果。

- 计算损失-通过比较网络输出和实际标签，计算损失函数的值，用于衡量模型预测的准确度。
- 反向传播-根据损失函数的梯度，利用反向传播算法更新网络参数，以减小损失。
- 迭代-重复以上步骤，直到达到预定义的停止条件（如达到最大迭代次数或损失函数收敛）为止。

3. 实验结果

整个流程包括生成数据、校正、验证并可视化结果三部分。

```
# Main
# 1. 生成数据
whitelist = generate_whitelist_umis(n_umis=100, length=12)
noisy_data = simulate_pcr_and_sequencing(
    whitelist,
    n_reads_to_sequence=10000,
    n_initial_molecules=1000,
    pcr_cycles=10,
    seq_error_rate=0.005,
    indel_rate=0.01
)

# 2. 校正过程
final_results, training_history = run_optimal_pipeline(noisy_data,
whitelist)

# 3. 验证并可视化结果
validate_and_visualize(final_results, noisy_data, whitelist,
training_history)
```

3.1. 输入数据

由于在各大单细胞数据库中找到的 UMI 矩阵基本都被处理过，而原始 FASTQ 文件又过于巨大，于是本实验中仿照提取 UMI 的原理，首先随机生成了多段长度一致的彼此独立的 UMI，再对其进行模拟 PCR 过程，引入突变。

3.1.1. 生成白名单

首先，通过 generate_whitelist_umis 函数生成一个包含 100 种独特 UMI 的白名单，每个 UMI 长度为 12 个碱基，且任意两个 UMI 之间的汉明距离不小于 3，这代表了理论上存在的、无错误的 UMI 模板集合。

- 附：由于电脑运行速度有限，故设置样本量较小，可根据需求调整

```
def generate_whitelist_umis(n_umis=100, length=12, min_hamming=3):
    """生成一个已知正确的UMI白名单，确保序列之间有足够的差异。"""
    logger.info(f"Generating a whitelist of {n_umis} UMIs
(length={length})...")
    bases = ['A', 'T', 'C', 'G']
```

```

umis = []

for _ in tqdm(range(n_umis), desc="Generating Whitelist"):
    while True:
        umi = ''.join(random.choice(bases) for _ in range(length))
        if not umis or all(sum(c1 != c2 for c1, c2 in zip(umi, u)) >=
min_hamming for u in umis):
            umis.append(umi)
            break
    return umis

```

3.1.2. 模拟 pcr 及扩增

随后，基于此白名单，通过 `simulate_pcr_and_sequencing` 函数模拟了一个包含 20,000 条最终测序读段的文库。该过程分层引入了两类错误：首先，通过模拟 12 轮 PCR 扩增（扩增效率为 85%），复现了突变的传播与累积；接着，在从扩增产物池中抽样后，对每个读段额外引入了 0.5% 的测序替换突变率和 1% 的插入/缺失（Indel）突变率。最终生成的数据集为后续的校正流程提供了包含各种复杂突变模式、且带有真实来源标签（Ground Truth）的高质量输入。

```

def simulate_pcr_and_sequencing(
    whitelist,
    n_reads_to_sequence=50000,
    n_initial_molecules=5000,
    pcr_cycles=12,
    pcr_efficiency=0.85,
    pcr_error_rate=1e-4,
    seq_error_rate=0.005,
    indel_rate=0.01
):
    logger.info(f"Simulating PCR & Sequencing for {n_reads_to_sequence}
final reads...")

    umi_length = len(whitelist[0])
    bases = ['A', 'T', 'C', 'G']

    #步骤 1: 初始分子捕获
    initial_molecules = random.choices(whitelist, k=n_initial_molecules)

    #步骤 2: PCR 扩增 (引入PCR错误)
    final_pcr_pool = []
    for original_umi in tqdm(initial_molecules, desc="Simulating PCR
Families"):
        family = [list(original_umi)]
        for _ in range(pcr_cycles):
            newly_synthesized = []
            for template in family:
                if random.random() < pcr_efficiency:

```

```

        new_strand = list(template)
        for i in range(len(new_strand)):
            if random.random() < pcr_error_rate:
                new_strand[i] = random.choice([b for b in
bases if b != new_strand[i]])
            newly_synthesized.append(new_strand)
        family.extend(newly_synthesized)
        if len(family) > 2000: family = random.sample(family, 2000)

    final_pcr_pool.extend(["".join(umi), original_umi) for umi in
family])
    logger.info(f"PCR simulation produced a pool of {len(final_pcr_pool)}
molecules.")

# 步骤 3: 测序 (引入测序错误)
if not final_pcr_pool: return []
num_to_sample = min(n_reads_to_sequence, len(final_pcr_pool))
sequencing_candidates = random.sample(final_pcr_pool,
k=num_to_sample)

final_noisy_data = []
for pcr_product, original_umi in tqdm(sequencing_candidates,
desc="Simulating Sequencing"):
    sequenced_read = list(pcr_product)
    # 测序替换错误
    for i in range(len(sequenced_read)):
        if random.random() < seq_error_rate:
            sequenced_read[i] = random.choice([b for b in bases if
b != sequenced_read[i]])
    # 测序Indel错误
    if random.random() < indel_rate:
        if random.random() < 0.5 and len(sequenced_read) > 1: del
sequenced_read[random.randint(0, len(sequenced_read) - 1)]
        else: sequenced_read.insert(random.randint(0,
len(sequenced_read)), random.choice(bases))

    noisy_umi_str = "".join(sequenced_read)
    # 模拟质量分数
    qualities = [random.randint(10, 25) if (i < 2 or i >
len(noisy_umi_str) - 3) else random.randint(30, 40) for i in
range(len(noisy_umi_str))]
    final_noisy_data.append((noisy_umi_str, qualities, original_umi))

return final_noisy_data

```

3.2. 校正流水线

结合前文已述的三类方法，本研究设计并实现了一个三阶段混合式流水线，旨在高效、准确地校正因 PCR 扩增和测序过程引入突变的读段。

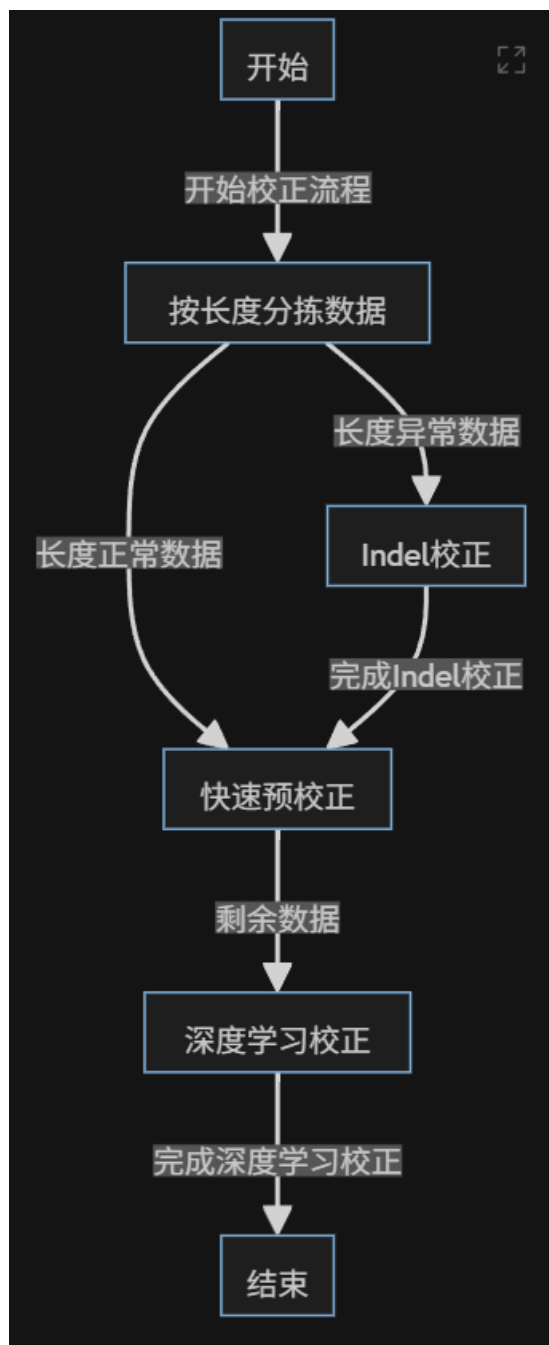


图 1 校正流程图

3.2.1. 阶段一：靶向 Indel 校正

此阶段专门处理在初始分拣中被识别出的、由于插入或缺失导致的长度异常序列。采用经典的 Smith-Waterman 局部比对算法，计算每个变长序列与白名单中所有正确序列的比对得分。通过选择得分最高的白名单序列作为校正结果（需超过一个动态设定的得分阈值），该阈值根据白名单 UMI 长度动态设定（例如，代码中设为完美匹配分值的 70%）。如果最高分仍未达到该阈值，则该序列将被视为无法校正。

```

# 使用Smith-Waterman算法处理长度异常的序列
def smith_waterman(seq1, seq2, match=2, mismatch=-1, gap=-2):
    m, n = len(seq1), len(seq2)
    score = np.zeros((m+1, n+1))
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            sc = match if seq1[i-1] == seq2[j-1] else mismatch
            score[i,j] = max(0, score[i-1,j-1] + sc, score[i-1,j] + gap,
                             score[i,j-1] + gap)
    return np.max(score)

def stage3_indel_correction(variable_length_data_with_indices,
                             whitelist):
    logger.info("Running Stage 3: Targeted Indel Correction...")
    corrected_results = {}
    umi_length = len(whitelist[0])
    score_threshold = umi_length * 2 * 0.7

    for i, umi, qual, original in tqdm(variable_length_data_with_indices,
                                         desc="Stage 3 (Indels)"):
        scores = [smith_waterman(umi, wl_umi) for wl_umi in whitelist]
        best_score_idx = np.argmax(scores)

        if scores[best_score_idx] >= score_threshold:
            corrected_results[i] = (whitelist[best_score_idx], original,
                                    "Indel-Corrected")
        else:
            corrected_results[i] = (None, original, "Indel-Uncorrected")

    return corrected_results

```

3.2.2. 阶段二：预校正

此阶段以最高效率处理绝大多数无突变或仅包含单个替换突变的序列。首先对白名单预先构建一个“1-汉明距离索引”哈希表。对于每一个长度正常的输入序列，算法首先进行精确匹配，若失败则在该索引中进行 $O(L)$ 复杂度的快速查找。超过 80% 的简单突变可以在此阶段被迅速解决，剩余的复杂突变的无法匹配序列则被传递至下一阶段。

```

def build_1_hamming_index(whitelist):
    """用于快速查找汉明距离为1的变体。"""
    logger.info("Building 1-Hamming distance index...")
    index = {}
    bases = ['A', 'T', 'C', 'G']
    for umi in tqdm(whitelist, desc="Building Index"):
        for i in range(len(umi)):
            original_char = umi[i]
            for base in bases:

```

```

        if base != original_char:
            variant = umi[:i] + base + umi[i+1:]
            # 如果一个变体可能来自多个白名单UMI，标记为歧义 (None)
            index[variant] = umi if variant not in index else
None
        return index

def stage1_fast_correction(noisy_data_with_indices, whitelist,
                           hamming_index):
    """阶段一：使用精确匹配和汉明索引进行快速校正。"""
    logger.info("Running Stage 1: Fast Pre-correction...")
    whitelist_set = set(whitelist)
    corrected_results = {}
    remaining_data = []

    for i, umi, qual, original in tqdm(noisy_data_with_indices,
                                        desc="Stage 1"):
        if umi in whitelist_set:
            corrected_results[i] = (umi, original, "Exact Match")
        elif umi in hamming_index and hamming_index[umi] is not None:
            corrected_results[i] = (hamming_index[umi], original, "1-
Hamming")
        else:
            remaining_data.append((i, umi, qual, original))

    return corrected_results, remaining_data

```

3.2.3. 阶段三：深度学习算法辅助校正

此阶段专门处理在初始分拣后，那些长度正确但因替换突变未能匹配白名单的 UMI。设计训练 1D 卷积去噪自编码器，通过卷积层和全连接层对 UMI 序列进行编码和解码，学习去噪表示。采用一个预训练的深度学习分类器，将校正问题重构为一个多分类任务，其中每个白名单序列代表一个独立的类别。该分类器直接预测待校正 UMI 最可能归属的白名单类别。通过选择模型输出置信度最高的白名单序列作为校正结果，从而完成对 UMI 的修正。

```

def one_hot_encode(umi, umi_length):
    base_map = {'A': 0, 'T': 1, 'C': 2, 'G': 3}
    encoded = np.zeros((umi_length, 4), dtype=np.float32)
    for i, base in enumerate(umi):
        if i < umi_length and base in base_map:
            encoded[i, base_map[base]] = 1
    return encoded.flatten()

class DenoisingConvAE(nn.Module):
    """1D Convolutional Denoising Autoencoder for UMI sequences."""
    def __init__(self, seq_len, n_features=4):
        super().__init__()

```

```

        self.seq_len, self.n_features = seq_len, n_features
        self.encoder = nn.Sequential(
            nn.Conv1d(n_features, 16, 3, padding=1), nn.ReLU(),
nn.BatchNorm1d(16),
            nn.Conv1d(16, 32, 3, padding=1), nn.ReLU(),
nn.BatchNorm1d(32),
            nn.Flatten(), nn.Linear(32 * seq_len, 32))
        self.decoder = nn.Sequential(
            nn.Linear(32, 32 * seq_len), nn.ReLU(),
            nn.Unflatten(1, (32, seq_len)),
            nn.ConvTranspose1d(32, 16, 3, padding=1), nn.ReLU(),
nn.BatchNorm1d(16),
            nn.ConvTranspose1d(16, n_features, 3, padding=1),
nn.Softmax(dim=1))

    def forward(self, x):
        x = x.reshape(-1, self.n_features, self.seq_len)
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded.reshape(-1, self.n_features * self.seq_len)

    def encode(self, x):
        with torch.no_grad():
            x = x.reshape(-1, self.n_features, self.seq_len)
            return self.encoder(x)

def train_autoencoder(model_class, whitelist, epochs=30, batch_size=128,
num_samples=20000, test_size=0.2):
    logger.info("Starting Classifier training with Validation Set...")

    noisy_data = simulate_errors(whitelist, n_reads=num_samples,
error_rate=0.05, indel_rate=0)

    # 输入(X): one-hot编码的带噪UMI
    umi_length = len(whitelist[0])
    X = np.array([one_hot_encode(u, umi_length) for u, __, _ in noisy_data])

    # 目标(y): 无噪UMI在白名单中的索引 (类别标签)
    whitelist_map = {umi: i for i, umi in enumerate(whitelist)}
    y = np.array([whitelist_map[o] for __, __, o in noisy_data])

    X_train, X_val, y_train, y_val = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y
    )

    X_train_t = torch.tensor(X_train, dtype=torch.float32)

```

```

y_train_t = torch.tensor(y_train, dtype=torch.long)
X_val_t = torch.tensor(X_val, dtype=torch.float32)
y_val_t = torch.tensor(y_val, dtype=torch.long)

train_dataset = torch.utils.data.TensorDataset(X_train_t, y_train_t)
train_loader = torch.utils.data.DataLoader(train_dataset,
batch_size=batch_size, shuffle=True)

# 训练
input_dim = umi_length * 4
num_classes = len(whitelist)
model = model_class(input_dim, num_classes)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

history = {'train_loss': [], 'val_loss': [], 'val_accuracy': []}

logger.info(f"Training on {len(X_train)} samples, validating on
{len(X_val)} samples.")

for epoch in range(epochs):
    model.train()
    epoch_loss = 0
    for noisy_batch, clean_labels_batch in train_loader:
        optimizer.zero_grad()
        outputs = model(noisy_batch)
        loss = criterion(outputs, clean_labels_batch)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    avg_train_loss = epoch_loss / len(train_loader)
    history['train_loss'].append(avg_train_loss)

    model.eval()
    with torch.no_grad():
        val_outputs = model(X_val_t)
        val_loss = criterion(val_outputs, y_val_t).item()
        _, predicted = torch.max(val_outputs.data, 1)
        total = y_val_t.size(0)
        correct = (predicted == y_val_t).sum().item()
        val_accuracy = 100 * correct / total

    history['val_loss'].append(val_loss)
    history['val_accuracy'].append(val_accuracy)

```

```

        if (epoch + 1) % 5 == 0 or epoch == 0:
            logger.info(f"Epoch [{epoch+1}/{epochs}], Train Loss:
{avg_train_loss:.4f}, Val Loss: {val_loss:.4f}, Val Accuracy:
{val_accuracy:.2f}%")

    logger.info("Classifier training finished.")
    return model, history

def stage2_dl_correction(remaining_data_with_indices, model, whitelist):
    logger.info("Running Stage 2: Deep Learning Correction...")

    model.eval()
    umi_length = len(whitelist[0])

    corrected_results = {}
    for i, umi, qual, original in tqdm(remaining_data_with_indices,
desc="Stage 2 (DL)":
        # 1. 将单个UMI编码为张量
        umi_tensor = torch.tensor(one_hot_encode(umi, umi_length),
dtype=torch.float32).unsqueeze(0)

        # 2. 使用模型直接预测最可能的类别（即白名单中的索引）
        with torch.no_grad():
            output = model(umi_tensor)
            _, predicted_idx = torch.max(output.data, 1)
            best_match_idx = predicted_idx.item()

        # 3. 从白名单中获取校正结果
        corrected_results[i] = (whitelist[best_match_idx], original, "DL-
Corrected")

    return corrected_results

class UMIClassifier(nn.Module):
    def __init__(self, input_dim, num_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.BatchNorm1d(256),
            nn.Dropout(0.5),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.BatchNorm1d(128),

```

```

        nn.Linear(128, num_classes)
    )
    def forward(self, x):
        return self.net(x)

```

3.2.4. 全流程校正

根据图 1 所示流程图，进行完整的三阶段 UMI 校正。

```

def run_optimal_pipeline(noisy_data, whitelist):
    umi_length = len(whitelist[0])
    logger.info(" Starting Optimal UMI Correction Pipeline ")

    # 按长度分拣数据，并保留原始索引i
    normal_length_data = []
    variable_length_data = []
    for i, data_point in enumerate(noisy_data):
        if len(data_point[0]) == umi_length:
            normal_length_data.append((i, *data_point))
        else:
            variable_length_data.append((i, *data_point))

    logger.info(f"Sorted data: {len(normal_length_data)} normal length,
{len(variable_length_data)} variable length.")

    all_corrected_results = {}
    start_time = time.time()

    # 阶段一：Indel 校正
    if variable_length_data:

all_corrected_results.update(stage3_indel_correction(variable_length_data,
whitelist))

    # 阶段二：快速预校正
    if normal_length_data:
        hamming_index = build_1_hamming_index(whitelist)
        stage1_corrected, stage1_remaining =
stage1_fast_correction(normal_length_data, whitelist, hamming_index)
        all_corrected_results.update(stage1_corrected)

    # 阶段三：深度学习校正
    if stage1_remaining:
        trained_model, training_history =
train_autoencoder(UMIClassifier, whitelist)

all_corrected_results.update(stage2_dl_correction(stage1_remaining,
trained_model, whitelist))

```

```

pipeline_time = time.time() - start_time
logger.info(f" Pipeline finished in {pipeline_time:.2f} seconds ")

# 重新排序，保持与输入一致
final_results = [None] * len(noisy_data)
for i in range(len(noisy_data)):
    if i in all_corrected_results:
        final_results[i] = all_corrected_results[i]
    else:
        final_results[i] = (None, noisy_data[i][2], "Uncorrected")

return final_results, training_history if 'training_history' in
locals() else None

```

3.3. 校正结果

计算指标并生成可视化图表来验证校正效果。

1. 计算核心指标

通过计算两个关键指标来量化评估整个 UMI 校正流程的性能。计算校正召回率，衡量被成功校正的 UMI 所占的比例，反映算法的覆盖广度。计算校正准确率，仅在已被成功校正的 UMI 子集上进行评估，衡量这些校正结果与它们的原始真实序列完全一致的比例，反映算法的精确度。二者共同验证校正算法的有效性。

```

def validate_and_visualize(final_results, noisy_data, whitelist,
training_history=None):
    original_umis = [item[2] for item in noisy_data]
    corrected_umis = [item[0] for item in final_results]

    accuracy = accuracy_score(
        [o for i, o in enumerate(original_umis) if corrected_umis[i] is
not None],
        [c for c in corrected_umis if c is not None]
    )

    total_reads = len(noisy_data)
    corrected_count = total_reads - corrected_umis.count(None)
    recovery_rate = corrected_count / total_reads

    logger.info(" Validation Metrics ")
    logger.info(f"Recovery Rate (能被校正的比例): {recovery_rate:.2%}")
    logger.info(f"Accuracy (在被校正的部分中，校对正确的比例): {accuracy:.2%}")

```

```

2025-06-19 19:26:49,461 - INFO - Recovery Rate (能被校正的比例): 100.00%
2025-06-19 19:26:49,462 - INFO - Accuracy (在被校正的部分中，校对正确的比例): 100.00%

```

图2 召回率与准确率

如图 2 所示结果，该算法在小数据量（100 个白名单序列）中召回率与准确率均达到 100%，说明算法效果十分可观。

2. 可视化校正前后的汉明距离分布

```
pre_correction_distances = []
umi_len = len(whitelist[0])
for noisy, _, original in noisy_data:
    if len(noisy) == umi_len:
        pre_correction_distances.append(sum(c1 != c2 for c1, c2 in
zip(noisy, original)))

# 计算校正后，校正结果与其真实来源的汉明距离
post_correction_distances = []
for corrected, original, _ in final_results:
    if corrected and len(corrected) == umi_len:
        post_correction_distances.append(sum(c1 != c2 for c1, c2 in
zip(corrected, original)))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
sns.countplot(x=pre_correction_distances, ax=ax1, palette='viridis')
ax1.set_title('Before Correction: Hamming Distance to True UMI',
fontsize=16)
ax1.set_xlabel('Hamming Distance', fontsize=12)
ax1.set_ylabel('Number of Reads', fontsize=12)
ax1.set_yscale('log')

sns.countplot(x=post_correction_distances, ax=ax2, palette='magma')
ax2.set_title('After Correction: Hamming Distance to True UMI',
fontsize=16)
ax2.set_xlabel('Hamming Distance', fontsize=12)
ax2.set_ylabel('Number of Reads', fontsize=12)
ax2.set_yscale('log')

fig.suptitle('Effectiveness of UMI Correction Pipeline', fontsize=20,
y=1.02)
plt.tight_layout()
plt.show()
```

如图 3 所示结果，左图展示校正前汉明距离的分布，右图展示校正后汉明距离的分布。我们可以看到，校正前大部分读段的汉明距离为 0（约 10^4 个），说明它们本身就是正确的。同时，存在大量汉明距离为 1（约数千个）和少量汉明距离为 2 的读段，这些是包含一个或两个突变的待校正序列。而经过校正流程处理后，所有序列的汉明距离都变为 0，表明该 UMI 校正流程非常有效。它成功地识别并修正了那些汉明距离为 1 和 2 的含突变序列，将它们完美地校正回了与真实序列完全一致的状态。最终，所有读段都与它们的真实来源序列相匹配，极大地提升了测序的质量和准确性。

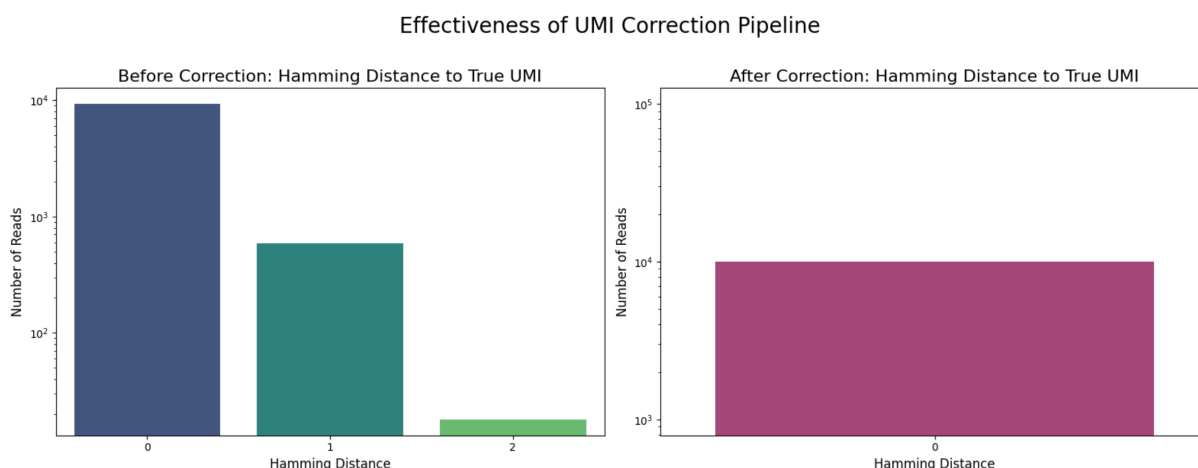


图3 校正前后汉明距离分布

3. 可视化序列数量变化

统计原始带噪数据及校正后数据中所有不同序列的种类数量，反映了由于测序错误而被人称为“夸大”的序列“多样性”。

```
pre_correction_counts = Counter([item[0] for item in noisy_data])
post_correction_counts = Counter(filter(None, corrected_umis))

fig, ax = plt.subplots(figsize=(10, 6))
bar_width = 0.35
labels = ['Unique UMI Species']
x = np.arange(len(labels))

ax.bar(x - bar_width/2, [len(pre_correction_counts)], bar_width,
label='Before Correction', color='skyblue')
ax.bar(x + bar_width/2, [len(post_correction_counts)], bar_width,
label='After Correction', color='salmon')

ax.set_ylabel('Number of Unique UMIs', fontsize=12)
ax.set_title('Reduction in UMI Complexity', fontsize=16)
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()

for i, count in enumerate([len(pre_correction_counts),
len(post_correction_counts)]):
    ax.text(x[0] + (i-0.5)*bar_width, count + 50, str(count),
ha='center', va='bottom', fontsize=12)
```

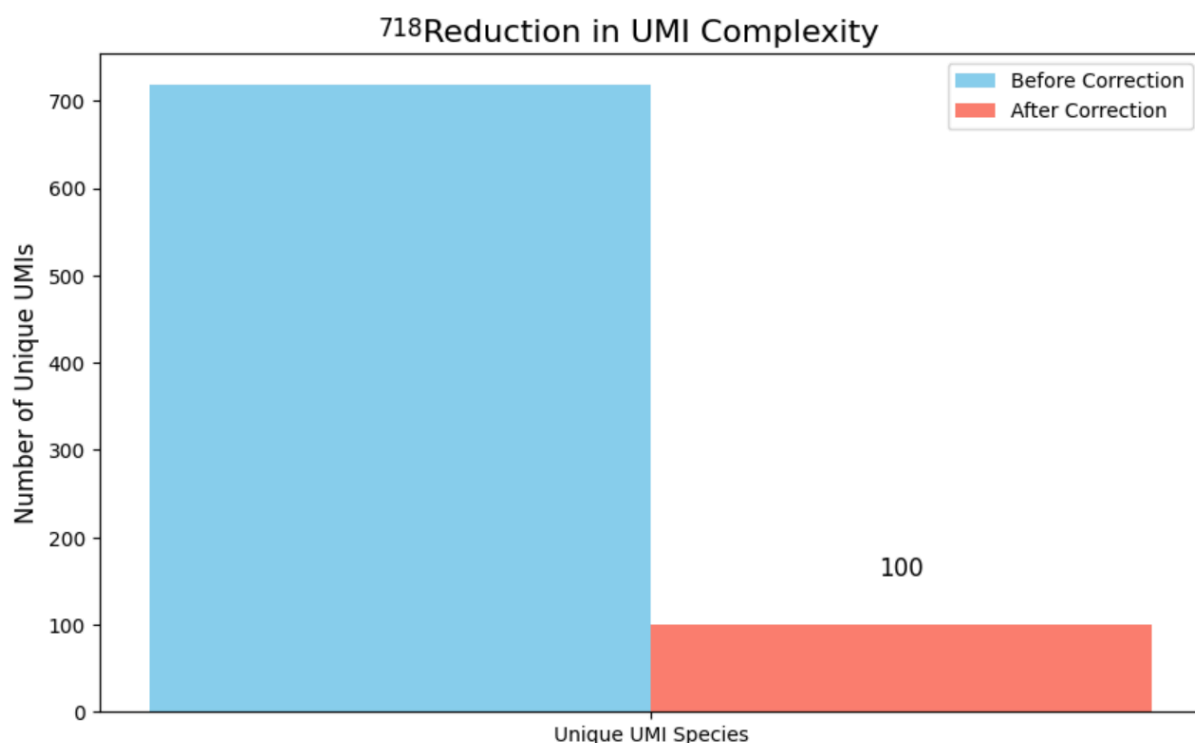


图4 序列多样性

如图4所示结果，带噪数据有718种不同的序列，远远大于一开始生成的UMI白名单，而校正处理后又回归了原始的100个，即实验设计中理论上存在的真实UMI种类数量，说明校正效果很好，完美修正了序列的突变并对应到原有的白名单。

4. 可视化神经网络训练过程

记录神经网络训练中每个epoch的Loss和Accuracy并可视化，反映其拟合效果。

```
if training_history:
    fig, ax1 = plt.subplots(figsize=(12, 6))

    color = 'tab:red'
    ax1.set_xlabel('Epochs', fontsize=12)
    ax1.set_ylabel('Loss', color=color, fontsize=12)
    ax1.plot(training_history['train_loss'], color=color,
label='Train Loss')
    ax1.plot(training_history['val_loss'], color=color,
linestyle='--', label='Validation Loss')
    ax1.tick_params(axis='y', labelcolor=color)
    ax1.legend(loc='upper left')

    ax2 = ax1.twinx()
    color = 'tab:blue'
    ax2.set_ylabel('Accuracy (%)', color=color, fontsize=12)
    ax2.plot(training_history['val_accuracy'], color=color,
linestyle='--', label='Validation Accuracy')
```

```

ax2.tick_params(axis='y', labelcolor=color)
ax2.legend(loc='upper right')

fig.tight_layout()
plt.title('Training and Validation Metrics Over Epochs',
fontsize=16)
plt.show()
plt.show()

```

```

2025-06-19 21:34:12,657 - INFO - Training on 16000 samples, validating on 4000 samples.
2025-06-19 21:34:13,151 - INFO - Epoch [ 1/30], Train Loss: 1.3872, Val Loss: 0.1524, Val Accuracy: 99.70%
2025-06-19 21:34:14,871 - INFO - Epoch [ 5/30], Train Loss: 0.0292, Val Loss: 0.0144, Val Accuracy: 99.88%
2025-06-19 21:34:16,718 - INFO - Epoch [10/30], Train Loss: 0.0100, Val Loss: 0.0078, Val Accuracy: 99.90%
2025-06-19 21:34:18,549 - INFO - Epoch [15/30], Train Loss: 0.0055, Val Loss: 0.0055, Val Accuracy: 99.92%
2025-06-19 21:34:20,421 - INFO - Epoch [20/30], Train Loss: 0.0036, Val Loss: 0.0050, Val Accuracy: 99.95%
2025-06-19 21:34:22,331 - INFO - Epoch [25/30], Train Loss: 0.0058, Val Loss: 0.0064, Val Accuracy: 99.83%
2025-06-19 21:34:24,142 - INFO - Epoch [30/30], Train Loss: 0.0032, Val Loss: 0.0065, Val Accuracy: 99.85%
2025-06-19 21:34:24,144 - INFO - Classifier training finished.

```

图5 神经网络训练过程

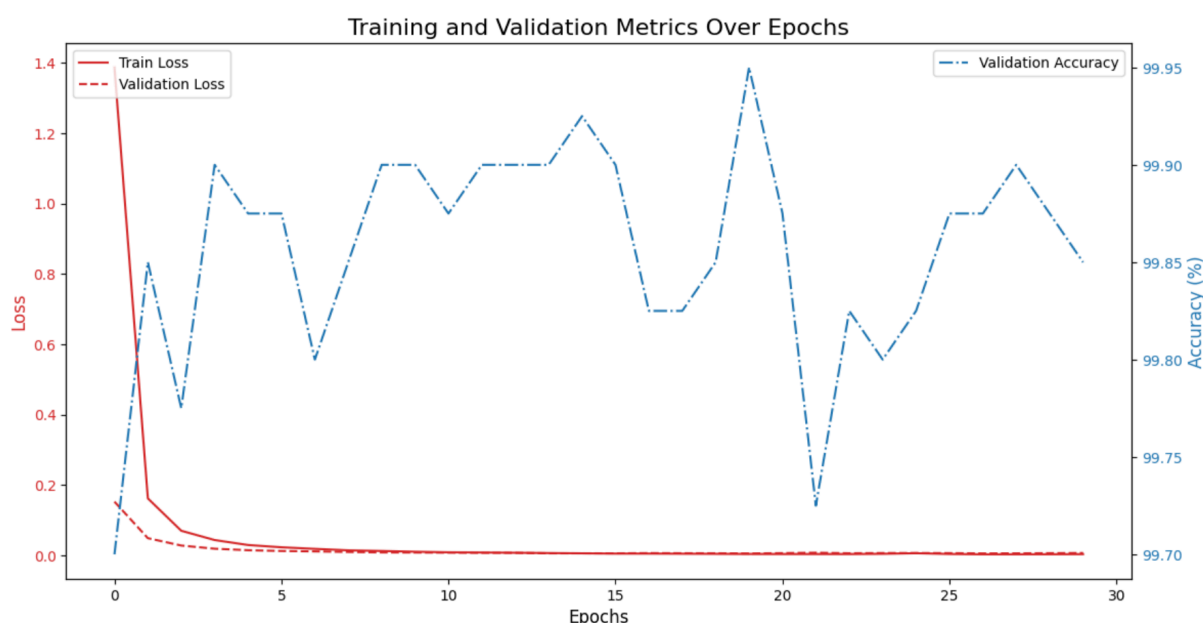


图6 神经网络算法效果

如图5和图6所示结果,可见该模型的准确率在训练集和验证集中均表现非常好,在99%以上, Loss 曲线也证明其学习效果好。但也存在过拟合,应设定早停机制进行优化。

3.4. 算法时间复杂度计算

1. 快速预校正

对于构建索引,假设白名单中有 N 个 UMI, 每个 UMI 的长度为 L 。对于每个 UMI, 需要遍历其每个碱基 (L 个), 对于每个碱基, 会尝试替换为其余 3 种碱基, 因此对于每个 UMI, 需要进行 $4L$ 次操作。因此, 构建索引的总操作数为 $O(4NL)$, 简化为 $O(NL)$ 。

对于快速查找校正, 假设带噪数据中有 M 个 UMI。对每个读段, 在白名单中查找精确匹配, 或者在汉明索引中查找, 可以通过哈希表在 $O(1)$ 时间内完成。因此, 此阶段的复杂度为 $O(M)$ 。

2. 深度学习校正

对于每个 UMI，进行 one-hot 编码需要 $O(L)$ 时间。因此，对于 M 个 UMI，总复杂度为 $O(ML)$ 。训练深度学习模型的复杂度取决于模型的结构和训练的轮数。假设模型有 K 个参数，每轮训练需要处理 num_samples 个样本，训练 epochs 轮。每轮训练的复杂度可以认为是 $O(K \cdot \text{num_samples})$ ，因此总复杂度为 $O(K \cdot \text{num_samples} \cdot \text{epochs})$ 。

对于每个剩余的 UMI，模型进行一次前向传播，复杂度为 $O(K)$ 。因此，对于 M 个 UMI，总复杂度为 $O(MK)$ 。

3. Indel 校正

Smith-Waterman 算法的复杂度为 $O(L^2)$ ，其中 L 是序列的长度。对于每个长度异常的 UMI，我们需要与白名单中的每个 UMI 进行比较，因此对于 M 个 UMI，总复杂度为 $O(MNL^2)$ 。

4. 总体复杂度

将上述各阶段的复杂度相加，总体复杂度大致为：

$$O(NL + M + ML + K \cdot \text{num_samples} \cdot \text{epochs} + MK + MNL^2)$$

其中， N 是白名单 UMI 的数量， M 是带噪 UMI 的数量， L 是 UMI 的长度， K 是模型参数的数量， num_samples 是训练样本的数量， epochs 是训练轮数。

4. 讨论

本实验旨在解决 UMI（唯一分子标识）两端测序不准的问题，采用了经典序列局部匹配、白名单汉明距离索引、深度学习等方法，取得了显著效果。但受限于电脑运行内存，数据规模远小于真正的单细胞测序数据，需要更大、更真实的数据集进行算法效果验证。

此外，关于 UMI 修正，已经有成熟的工具 UMI-tools[2]，引入基于网络的方法来考虑 UMI 序列中的错误，在识别 PCR 重复时加以处理。

5. 参考文献

[1]Karst, S.M., Ziels, R.M., Kirkegaard, R.H. et al. High-accuracy long-read amplicon sequences using unique molecular identifiers with Nanopore or PacBio sequencing. Nat Methods 18, 165–169 (2021).

[2]Smith T, Heger A, Sudbery I. UMI-tools: modeling sequencing errors in Unique Molecular Identifiers to improve quantification accuracy. Genome Res. 2017 Mar;27(3):491-499. doi: 10.1101/gr.209601.116. Epub 2017 Jan 18. PMID: 28100584; PMCID: PMC5340976.

附注：本实验在模拟 pcr 扩增引入突变和深度学习部分有借助 AI 进行 debug